

P&D: Building a Receiver chain for GMSK modulation

Sjouke Spijkerman

ESAT

KU Leuven

Leuven, Belgium

sjouke.spijkerman@student.kuleuven.be

Thomas Debelle

ESAT

KU Leuven

Leuven, Belgium

thomas.debelle@student.kuleuven.be

Alen Komadina

ESAT

KU Leuven

Leuven, Belgium

alen.komadina@student.kuleuven.be

Josip Mihaljević

ESAT

KU Leuven

Leuven, Belgium

josip.mihaljevic@student.kuleuven.be

Abstract—This project designed a Gaussian Minimum Shift Keying (GMSK) receiver chip featuring a Programmable-Gain Amplifier (PGA), flash Analog-To-Digital Converter (ADC), and digital demodulation blocks. Subblocks met the 17.8 dB Signal-to-Noise Ratio (SNR) requirement for level 7, the highest achievable without Forward Error Correction (FEC) and integrated reached level 6. A full mixed-signal simulation was also successfully run. Starting from a high-level Python model, the analog, mixed-signal, and digital components were developed and implemented using industry-standard tools to improve bit error rates in noisy conditions.

I. HIGH-LEVEL MODEL

Signals degrade during transmission due to channel effects, which add noise and reduce signal amplitude. While receivers can handle minor decoding errors, pre-processing the signal before decoding improves power efficiency and accuracy. A typical GMSK receiver has an analog frontend including an antenna, downmixer, filter, an Automatic Gain Controller (AGC), and an ADC that converts the signal for the digital backend. The digital backend includes an IQ downmixer, GMSK demodulate, clock recovery, bit extractor, and decoder. The overall system architecture is shown in Fig. 1 and will be developed and prototyped starting with a high-level model.

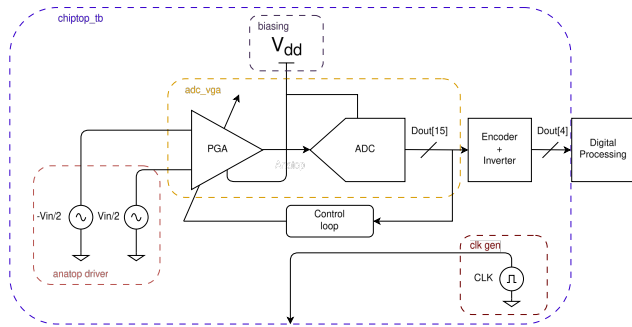


Fig. 1. A High-Level Overview of the GMSK-receiver

A. AGC

Although GMSK encodes information in phase rather than amplitude, the ADC output must resemble a sine wave to accurately extract phase information. To optimize phase extraction, the signal amplitude must fully utilize the ADC's Dynamic Range (DR). This is ensured by an AGC, which uses a feedback loop from the ADC to adjust amplification so the

signal spans the DR precisely.

AGC implementations are typically classified as PGA or Variable Gain Amplifier (VGA). A PGA offers discrete gain steps, requiring a trade-off between the number of levels and gain accuracy. In contrast, VGAs provide continuous gain via analog control loops but are more complex and prone to stability issues [1]. Therefore a PGA was chosen for this design.

For gain adjustment peak detection is performed. The signal being centered at a common mode voltage of $V_{dd}/2$, peak detection monitors the maximum amplitude per period to decide whether to increase or maintain gain.

B. ADC

For a GMSK receiver a wide choice of ADCs exists. Unlike other ADC architectures such as a Successive-Approximation Register (SAR) ADC, flash ADCs allow immediate conversion of the analog input by leveraging parallel comparison. A flash ADC was chosen due to its straightforward design and high-speed operation. The core comparator is a StrongARM latch, which offers rail-to-rail output for a clean digital transition. To hold the output state when the clock is low, a NAND-based latch follows the comparator [2].

C. IQ Downmixer

An *IQ downmixer* converts high-frequency modulated signals to baseband for efficient demodulation and processing. It multiplies the input with a Local Oscillator (LO) to generate in-phase (*I*) and quadrature (*Q*) components. This is achieved by mixing the received signal $s(t)$ with sine and cosine at the carrier frequency f_c , yielding:

$$I(t) = s(t) \cos(2\pi f_c t), \quad Q(t) = s(t) \sin(2\pi f_c t) \quad (1)$$

A Numerically Controlled Oscillator (NCO) ensures phase-continuous sine and cosine generation. While exact sine and cosine waves can be used, the *CORDIC algorithm* provides an optimized way to compute these values, reducing computational complexity.

After mixing, the resulting *I/Q signals* still contain high-frequency components, which must be removed using a *low-pass filter (LPF)* to eliminate unwanted noise and aliasing. In this implementation, a *Cascaded Integrator-Comb (CIC) filter* is used for efficient down-sampling and filtering, as it avoids multiplications. The CIC filter effectively reduces the sampling

rate while maintaining signal integrity, preparing the complex envelope for subsequent demodulation.

To further process the signal, the phase information can be extracted using the $\text{atan2}(Q, I)$ function. This operation is critical for demodulation, as the phase carries the encoded information in phase-modulated schemes like GMSK. The *CORDIC algorithm* can also be employed here to compute the arctangent efficiently in hardware.

In sum, the IQ downmixer with filtering and phase extraction enables robust, hardware-efficient demodulation.

D. GMSK-demodulation

To enhance performance and robustness against noise a matched filtering approach can be applied before the phase extraction [3]. It exploits correlation via convolution, which can be computed efficiently in the frequency domain—ideal for VLSI using shift registers and multipliers.

This approach maximizes SNR, reducing Bit Error Rate (BER), as demonstrated in Fig. 2. Here, a simulated clock recovery introduces Gaussian noise ($\sigma = 100ps$), typical for basic recovery schemes.

An SNR of 17.8 dB was selected as the target limit for analog and digital stages.

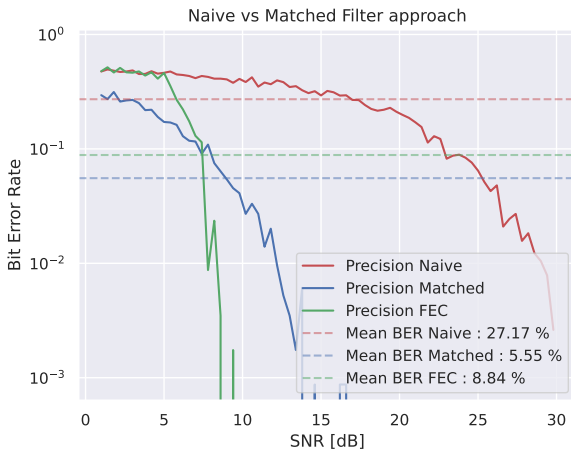


Fig. 2. BER for various SNR values

E. Clock recovery

Clock recovery synchronizes sampling to incoming data, compensating for noise, jitter, and frequency shifts [4]. In GMSK receivers, accurate clock recovery is essential for low BER, especially under low SNR and high interference.

Techniques for clock recovery include *Zero-Crossing Detection*, *Gardner Timing Error Detector (TED)*, *Maximum Likelihood (ML) TED* and *Phase-Locked Loop (PLL)-Based Clock Recovery* [5]. PLL-based recovery is well-suited for GMSK, which can dynamically adjust to jitter, drift, and noise. A PLL continuously refines its timing estimate, making it ideal for cases with low SNR and high interference. Additionally, an adaptive PLL facilitates fine-tuning of loop bandwidths, which enhances convergence speed and stability.

Since a PLL structure is cumbersome and costly to implement, a simpler approach was used. The method in question is the so called *Early-Late Gate Clock Recovery* [6]. It achieves approximately 0.1% deviation in clock timing, enabling reliable data decoding. Clock recovery performance is shown in Fig. 3.

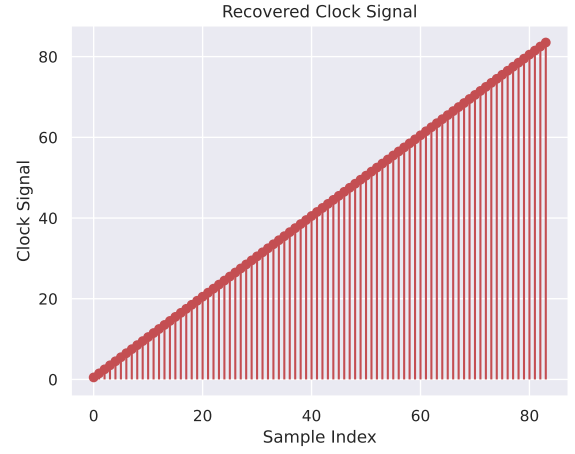


Fig. 3. Moments of the recovered clock.

II. SPECIFICATIONS

With the high-level architecture in place, the system can be specified in more detail.

A. Analog & Mixed-Signal

The input signal has a $20kHz$ carrier ($\pm 500Hz$) and amplitude between $1mV$ and $100mV$. Given its low and variable amplitude, appropriate amplification is essential to maximize signal swing prior to conversion.

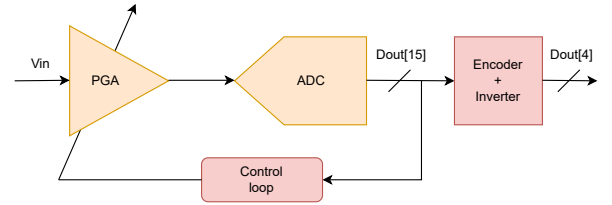


Fig. 4. Analog and Mixed-Signal building blocks

Fig. 4 outlines the analog and mixed-signal blocks required for GMSK. The following sections address the function, design, and optimization of each block.

1) *PGA*: The PGA specifications are listed in Table I.

Parameter	Description
Amplification range	13.06 – 53.06dB (1 mV–100 mV → 450 mV)
Bandwidth	20.5 kHz
SNR _{Analog + Downmixer}	17.8 dB
Power	Minimized; Same order of magnitude digital part

TABLE I
SYSTEM CHARACTERISTICS OVERVIEW

Amplification range and bandwidth follow project specifications [7]. Noise targets reflect the minimum SNR needed for digital processing without FEC. As this is only a prototype, power consumption is required to fall within the same order of magnitude as the digital section.

The AGC maximizes the ADC's dynamic range while preserving signal shape. A Miller OTA was chosen for its high gain and wide output swing, as speed is not critical. The OTA provides a 0.9V swing, limited by a 0.3V overdrive drop. Consequently, the smallest input amplitude of $1mV$ requires a gain of 450 to reach full use of DR. However, as signals of

only $\leq 1mV$ will need this amplification vector, a choice was made to lower the gain to 300 for the highest value. This value was chosen because $300mV$ is the lowest output voltage that allows the 10 gain stages to span evenly between $300mV$ and $450mV$, ensuring each stage operates within a usable output range. Likewise, for the maximum input amplitude of $100mV$, a gain of merely 4.5 is sufficient. This results in an overall gain variation ratio of:

$$\frac{300}{4.5} = 66.7 \quad (2)$$

Since full swing is not guaranteed, Eq. 3 includes a correction term:

$$SNR_{max,adc} = 6.02 \cdot N_{bits} + 1.76 - 20 \log_{10} \left(\frac{V_{dd}}{V_{pp}} \right) \quad (3)$$

For 4-bit resolution and a target SNR of 17.8, dB, the V_{dd}/V_{pp} maximum ratio is:

$$1.47 \leq \log_{10} \left(\frac{V_{dd}}{V_{pp}} \right) \quad (4)$$

From Eqs. 2 and 4, the minimum number of gain stages is calculated in Eq. 5:

$$N_{stages,4-bit} \geq \frac{\ln(66.7)}{\ln \left(\left(\frac{0.9}{1.2} \times 1.47 \right)^2 \right)} + 1 = \lceil 9.70 \rceil = 10 \quad (5)$$

The minimum gain needed in the Operational Transconductance Amplifier (OTA) was $55dB$ to get to a reasonably small static error, which may be adjusted with changing the resistor values afterwards.

2) *ADC*: Though 3 bits suffice theoretically, swing limitations reduce effective SNR. Therefore, a 4-bit ADC architecture was selected.

The Flash ADC comprises:

- 1) Comparator with NAND latch outputs
- 2) Resistive ladder

Fifteen comparators are used, each comparing input against a reference from the resistor string. A strongARM latch topology was selected for its low power, high speed, and minimal offset.

B. Digital

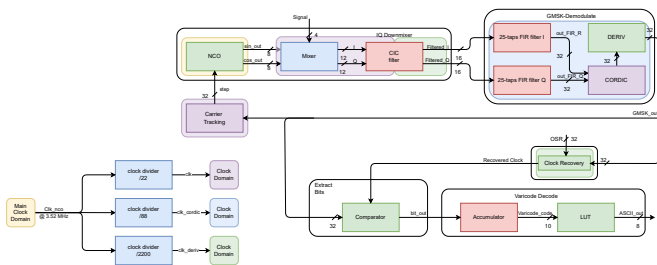


Fig. 5. High level model of Digital

Fixed-point formats Q16.16 (32-bit) and Q8.8 (16-bit) are used. Furthermore, the use of two's complement representation simplifies integration with default signed integer handling by SystemVerilog [8].

Multiple clock domains exist, the highest running at $3.52MHz$, as shown in Fig. 5.

1) *GMSK-Demodulation*: This block uses a $2 \cdot 16$ bits input and a $1 \cdot 32$ bits output. This choice of a smaller input was driven by the low ADC resolution. The bit width at the output was doubled to avoid truncation when doing multiplication. To conserve power, computationally intensive operations per cycle are avoided and the computational effort is spread over multiple clock cycle.

2) *Clock Recovery*: The clock recovery receives the 32-bit demodulated signal and a 32-bit Over-Sampling Ratio (OSR), with 16 as the default value.

To minimize power, complex operations are avoided in favor of simpler alternatives, such as using shifts instead of divisions.

III. REALIZATION

Now that the separate blocks have been further specified, a definite design can be introduced.

A. Analog

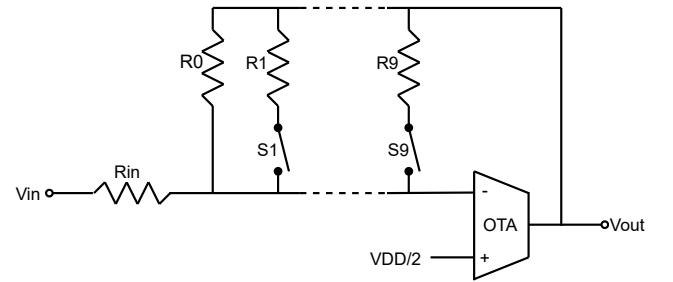


Fig. 6. The Programmable Gain Amplifier

1) *PGA*: The PGA topology, as depicted in Fig. 6, uses a resistive feedback network with an OTA. A Switch-Capacitor (SC)-based version was considered but not implemented due to time limits. The network includes resistors (R_0 – R_9) and switches (S_1 – S_9). With all switches off, only R_0 is in the loop, giving the maximum gain of 300, which maximizes feedback resistance. All gain levels are shown in table II.

Resistor	Value (real/ideal) (kΩ)	Gain (dB)	Switches On
R_{in}	1 / 1	/	/
R_0	580 / 300	300	none
R_1	485 / 503.5	188	S_1
R_2	300 / 317	118	S_1, S_2
R_3	195 / 198.5	74	S_1 to S_3
R_4	120 / 125	46.5	S_1 to S_4
R_5	75 / 77	29	S_1 to S_5
R_6	45 / 47.5	18	S_1 to S_6
R_7	26 / 32	11.5	S_1 to S_7
R_8	14.5 / 18	7	S_1 to S_8
R_9	9 / 12.5	4.5	S_1 to S_9

TABLE II
RESISTOR VALUES WITH CORRESPONDING GAIN VALUES.

Turning on each switch adds a resistor in parallel, reducing feedback resistance and lowering gain. This change follows an approximately linear 4dB step, as shown in Fig. 18.

Table II lists both ideal and real resistor values with their corresponding gains. Ideal values assume zero switch resistance ($R_{on,sw} = 0$) and infinite OTA gain.

The switch is a transmission gate made of parallel nMOS and pMOS transistors (Fig. 7). Their complementary behavior ensures low resistance when on. Both are sized $5 \times$ minimum width to reduce $R_{on,sw}$.

The Miller OTA has two stages: an input differential pair and a gain stage. The input transistors and transistor $M2$ (Fig. 8)

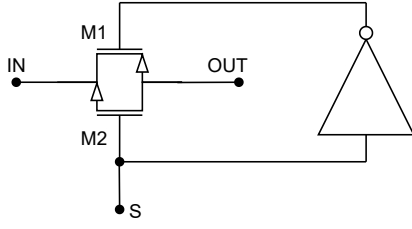


Fig. 7. Transmission gate as a switch

are kept minimal to reduce noise, improving SNR, especially important in low-voltage operation.

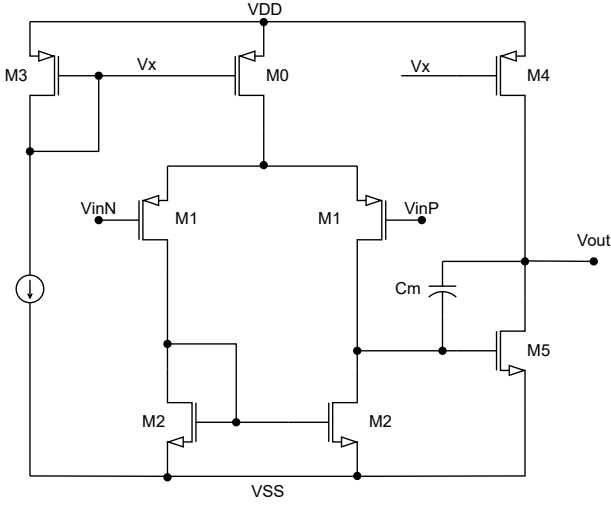


Fig. 8. The Miller OTA

The second stage uses larger transistors to prevent output clipping which was seen at low gain (4.5) and high input (100mV). Clipping did not occur at high gain (300) with low input (1mV). This change increased the DC current of OTA and the static power, but ensured better linearity and a higher output swing. This trade-off between power and performance was considered acceptable to guarantee signal accuracy by lowering the distortion level significantly at the gain stage with the lowest gain in the PGA.

The transistor sizes can be seen in Table III. The DC current source in the circuit is modeled as an ideal current source, supplying 15mA. The Miller capacitance, which is crucial for achieving the desired phase margin and bandwidth, is set to 1.5pF. This is elaborated on in Sec. IV.

Transistors	W/L
M0	12 μ m/500nm
M1	6 μ m/500nm
M2	5.9 μ m/500nm
M3	10 μ m/500nm
M4	70 μ m/400nm
M5	80 μ m/400nm

TABLE III
TRANSISTOR SIZES OF THE MILLER OTA

2) ADC: The topology, chosen for its low power, high speed, and low offset, is shown in Figure 9. It's adapted from a 45nm Complementary Metal-Oxide Semiconductor (CMOS)

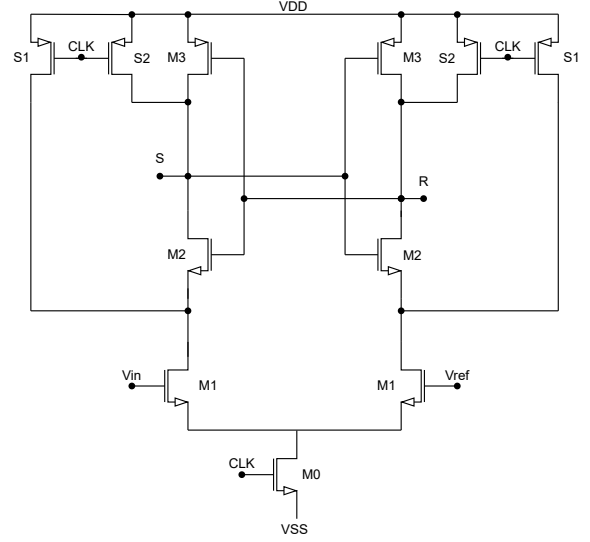


Fig. 9. The Strong-ARM Comparator

design [2] featuring strong positive feedback via cross-coupled latches for fast decisions. Transistor sizes (Table IV) were scaled for the 65nm technology used in this project, keeping the same aspect ratios.

Transistors	W/L
M0	240nm/60nm
M1	240nm/60nm
M2	280nm/60nm
M3	160nm/60nm
S1	160nm/60nm
S2	360nm/60nm

TABLE IV
TRANSISTOR SIZES OF THE STRONG-ARM COMPARATOR

The ADC generates a 15-bit thermometer code while the clock is high. A NAND latch from the course Design of Digital Integrated Circuits (DDIC) holds the output when the clock is low. A non-ratioed topology is chosen, which is shown in Fig. 10.

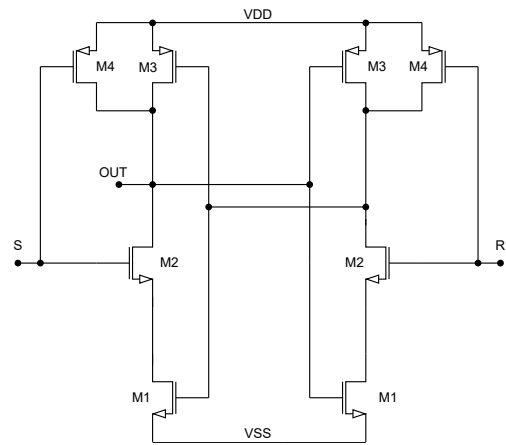


Fig. 10. The NAND latch

The sizes of the transistors of the NAND latch are the smallest possible in the used 65nm CMOS technology, which means that the lengths are 65nm and the widths 200nm.

B. Mixed Signals

1) *Control Loop*: The digital control loop monitors the ADC output to ensure the second-highest comparator triggers, indicating full use of the output swing of the VGA (0.9V). If the highest comparator triggers, gain can be reduced. However, since inputs never clip or vary significantly, no gain reduction was implemented.

To detect the signal peak accurately, a full waveform period is considered. At a 20kHz input and 160kHz clock, eight samples per period sufficiently capture the peak of a stable signal.

Gain changes are limited to once per signal period to prevent instability and avoid distortion.

2) *Inverter + Encoder*: The encoder converts the 16-bit thermometer code into a 4-bit binary value using a Look-Up Table (LUT)-based encoder, which feeds the digital control logic.

C. Digital

1) *IQ-Downmixer*: To realize the IQ downmixer, 4 crucial blocks are needed:

- 1) Mixer: for the coherent demodulation
- 2) NCO: to generate the sine and cosine
- 3) CIC filter: to go in band pass and decimate
- 4) Carrier recovery: to match precisely the carrier frequency

Each block will be addressed in their designated section. Here is an overall overview of the block:

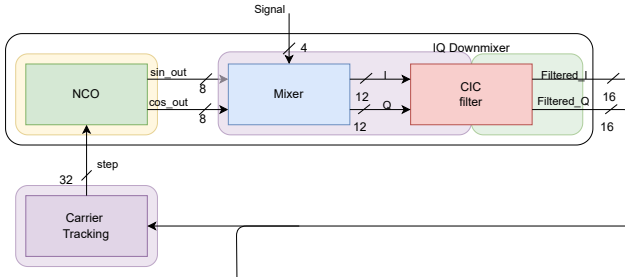


Fig. 11. IQ Downmixer in Hardware

a) *Mixer*: It is a simple multiplier where there are a 4 bit and an 8 bit input, which become a single 12 bit output. 4 bits come from ADC, and 8 bits come from the NCO. It is designed to complete the operation at every clock cycle of f_s .

b) *NCO*: The NCO is based on the CORDIC algorithm to generate the sine and cosine wave. Internally, it is working with 32 bit values that get rounded off afterwards to ensure a proper multiplication. It was decided to go for an 8 bit NCO as a slight improvement in the Python simulation was seen and the extra bits are obtained with a different trimming (Tab. IX).

c) *CIC filter*: For this one, an additional Python model was developed to better understand and measure the required precision and size needed. It was decided to work on a 3 stage CIC as it provided sufficient results compared to the ideal values, and without inducing too much overhead.

The Python and HDL-model shed light on an important problem of such filter, the overflow of intermediate value. In Hogenauer's filter, there exists an important formula to respect the bit width inside the filter:

$$\text{Bit Width} = \text{Bit Width}_{in} + \lceil \# \text{ stage} \cdot \log_2(DR) \rceil \quad (6)$$

Here, DR is the *downsampling ratio*. In the case of this demodulation scheme, it can be found:

$$DR = \left\lceil \frac{f_s}{OSR \cdot br} \right\rceil = 100 \quad (7)$$

f_s	160 kHz
OSR	16
Bit rate	100 bit/s

TABLE V
SYSTEM PARAMETERS

So, this means that every 1.6kHz , a new sample is generated and fed into the next block.

d) *Carrier Recovery*: The carrier recovery is based on heuristics observation. Typically, if the carrier guess is off by $\pm 100\text{Hz}$, the GMSK demodulated signal would be shifted up or down but would still be recognizable. So, the main idea was to first estimate the carrier frequency to be exact and if the signal was too high, an increase of the frequency was needed or vice versa.

However, if the signal was at the same time too high and too low, it was most likely a *gibberish* signal due to a higher error in f_c . In other words, the frequency guess was too far off to converge and another guess was needed. Each frequency guess is incremented by 200 Hz up to 19.5 kHz after which it loops back starting at 20.5 kHz.

This technique has been quite reliable even in noisy environment simulation. To avoid being perturbed by random spurs of noise, the carrier search is stopped after a few cycles since the carrier error is static, and not dynamic [7].

For more explanation about this see paragraph III-C2a.

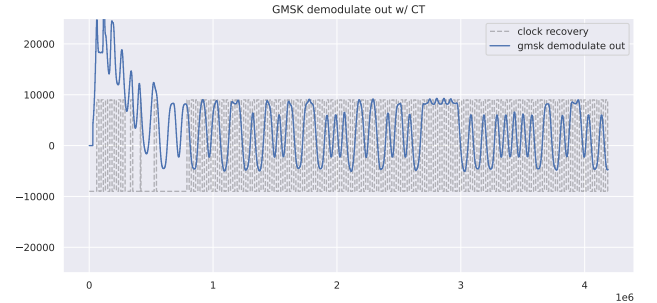


Fig. 12. Example of $f_c = 20.1\text{ kHz}$

2) *GMSK-demodulation*: The GMSK-demodulation needs 25 clock cycles to complete its work on one sample. Since a sample is produced at 1.6kHz (tab. V), a factor 25 times is needed to bring the clock up to 40kHz .

The delay results from the CORDIC algorithm used to find the phase based on the I and Q signals. Moreover, this block also implements some extra functions, such as the *unwrapping* the angle. A safety margin of *free* clock cycle is present to allow a possible hardware sharing between the two Finite Impulse Response (FIR) filters. Ultimately, this idea was not pursued.

This block is using 3 other sub-block types:

- 1) FIR: I:16b O:32b: a 25-taps FIR filter that is used as a convolution filter for the non-coherent demodulation scheme. The taps are based on the Gaussian function that is used to shape the signal at modulation. The implementation minimized the critical path.

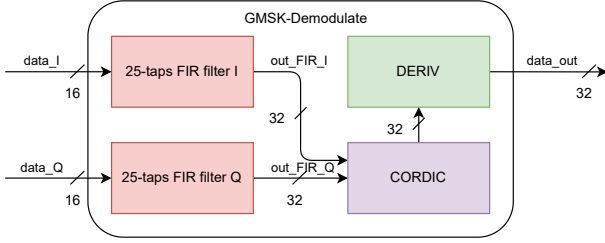


Fig. 13. GMSK-demodulation

- 2) **CORDIC**: I:2 · 32b O:32b: using the CORDIC algorithm to extract the phase of the complex number composed of our I and Q signals. This implements the *atan* operation.
- 3) **DERIV**: I:32b O:32b: to compute the difference between the current and previous received sample.

This implementation has been quite robust and the FIR filters are keys to reach low BER even in noisy environment, as proven in Fig. 2.

The implementation was straightforward, but debugging was challenging. Some issues were sudden spikes at the start of transmission, especially for low SNR input signal. This phenomenon was due to the delay from the input to the output of the FIR filter. Indeed, at first the filter is set to 0 for every register, but this causes a certainty that any noise will introduce larger error in the phase calculation creating, a sudden rotation. This was solved by letting the filter run for 10 cycles before activating the CORDIC and derivation block.

a) *Why does the carrier recovery work?*: No formal mathematical will be provided but a simple explanation will be given. Typically, if the coherent demodulation would be off by a few Hz, the demodulated signal will be composed of the desired signal and a slightly higher frequency (compared to desired one) residual sinusoidal wave. Those two signal will be forwarded and transmitted by the CIC filter as it will only filter out high frequencies.

The key for this method to work is the use of FIR filter. The filter will act as a correlation operation on the desired signal and as a moving average on the unwanted residual one. Due to this unwanted signal, the I and Q signal will rotate faster than expected or rather the rotation will be the sum of the information rotation (as *GMSK* scheme is based upon) and of a constant, faster, rotation that doesn't carry any information.

Then, the CORDIC block will behave as expected and will thus transcribe those *I* and *Q* coordinate in an angle. Finally this angle will be sent to the *deriv* block where it will find an unusual large difference on top of the wanted difference. This, resulting into the information signal on top of an offset as seen in fig. 12.

3) *Clock Recovery*: The clock recovery block may be seen in Fig. 14. As stated before, there are two inputs to this block, namely the OSR (6b) and the GMSK demodulated signal (32b). The single output is the recovered clock signal (1b).

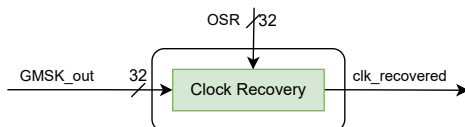


Fig. 14. Clock Recovery in Hardware

For the simplest implementation, the *blind oversampling* method was used. Sampling is done at the oversampling rate according to OSR, and then the center sample is selected based on a fixed offset. The logic behind this method is quite simple and easily synthesized, and it works well if the transmitter and receiver are closely synchronized. Nevertheless, the clock recovery block cannot track long-term clock drift, and this method is not optimal for environments with jitter or skew. The optimized clock recovery implementation uses the Early-Late Gate (ELG) method.

The ELG-method is a widely used non-linear timing recovery loop in digital receivers. The key steps of operation are: sampling the received signal at early, on-time, and late positions, computing the timing error as a difference between the late sample and the early sample, and adjusting the sampling clock dynamically based on the calculated error. This method introduces non-linear feedback, and it does not require an explicit phase detector, as the phase increment is computed dynamically. On the other hand, the convergence is slower compared to PLL-based recovery, and it is less effective in severe noise environments. The method in question is preferred over the PLL-based approach due to its simplicity, i. e. easier hardware development. Moreover, it does perform well for moderate noise and provides the necessary accuracy.

Key features of the design are the following:

- 1) OSR is configurable (e.g., 16, 32, etc.), so it works well with the desired one;
- 2) dynamic phase increment is computed automatically based on OSR, allowing the design to be flexible;
- 3) timing error correction is applied smoothly using a shift-based division, avoiding any costly division operations;
- 4) recovered clock output generates a single-cycle pulse exactly at the optimal sampling points (around or at peak values of the demodulated signal).

The module is naturally written in Verilog, and through simulation it was confirmed that the recovered clock aligns correctly with the center of the symbols after a short acquisition period. The recovered clock signal may be seen in Fig. 15.

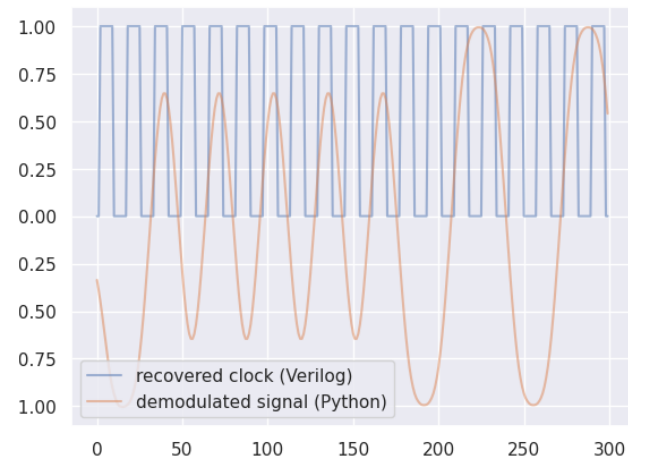


Fig. 15. Recovered clock signal using ELG-method

The total energy consumption per clock cycle is estimated to be around $0.6pJ$, and the total number of clock cycles needed for one period of the recovered clock signal would be 16 (the same as OSR). Hence, the total energy consumption per iteration would be approximately $9.6pJ$.

4) *Waveforms to ASCII*: The final blocks need to transform the GMSK-signal into an 8 bit wide ASCII character. For this, the recovered clock obtained at III-C3 and the GMSK signal from III-C2 is needed. If the carrier recovery converged as expected, the signal should be centered around 0, and so the decision algorithm can be as straightforward as:

$$\text{bit} (@ \text{ every clock pulse}) = \begin{cases} 1 & \text{if } sig > 0 \\ 0 & \text{else} \end{cases} \quad (8)$$

a) *Accumulate*: However, to decipher one *varicode* character, a maximum of 10 bits is needed. For this reason, there exists an *accumulate* block which saves each received bit into a buffer. After it has received 2 consecutive 0 [9], it signals to the next blocks that it has finished receiving one character.

Since this decoding is heavily influenced by transmission error (if it receives 01 instead of 00), some security features were added. Typically, the temporary buffer is made of 64 bits to avoid possible overflow in the case where it never receives the *new character* code (00). Moreover, this block can alert other blocks through a special wire *-error_detected-* and takes active care of the overflow error with certain *fail-safe hardware*.

b) *Varicode to ASCII*: The simplest way to implement it is by using a LUT generated by a Python script. The output of the block is an 8 bit one, and it represents a single character in ASCII.

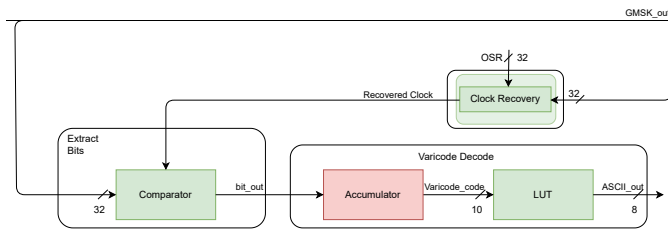


Fig. 16. Waveform to ASCII in Hardware

IV. TEST & VALIDATION

Before full system simulation, each block is tested individually. Afterwards, analog-mixed signal and digital tests are performed.

A. Analog

1) *Miller OTA*: The Miller OTA was tested in unity gain feedback. The main challenge was minimizing the output DC offset from the 600mV common-mode voltage, as any difference is amplified in the closed-loop PGA. An offset of 20μV was achieved, leading to a maximum PGA offset of 6mV at gain 300. Phase margin and gain results are shown in Figure 17.

2) *PGA*: The PGA results are shown in Table VI. The goal was an SNR above 17.8dB to support the digital section after the IQ downmixer. The minimum achieved SNR of 24.8dB exceeded this, ensuring flawless system operation.

After confirming sufficient SNR, the PGA performance can be verified, as shown in Fig. 18.

3) *Comparator & ADC*: The ADC shows significant deviation at the first step, causing high INL and DNL near the low input range due to the nMOS input pair at the first comparator operating below its 75mV reference. The delay caused is not affecting system performance, as the control loop

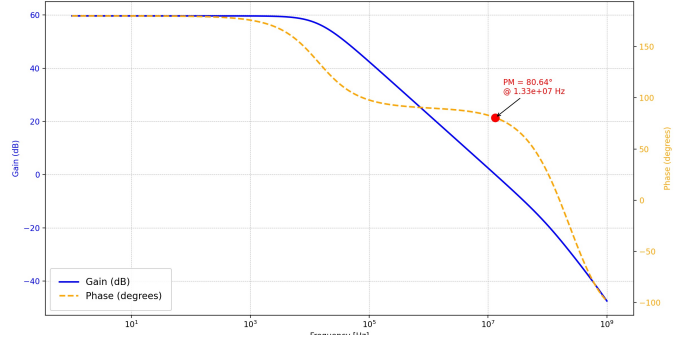


Fig. 17. AC simulation of the Miller OTA

Switches On	SNR (dB)	Gain (dB)
/	24.8	300
1	28.4	188
1, 2	32.5	118
1 to 3	36.3	74
1 to 4	40.1	46.5
1 to 5	43.7	29
1 to 6	47.1	18
1 to 7	50.3	11.5
1 to 8	57.1	7
1 to 9	59.7	4.5

TABLE VI
SWITCH CONFIGURATIONS WITH CORRESPONDING SNR AND GAIN VALUES.

is excluding the two highest and lowest Least-significant bit (LSB)s, keeping the Miller OTA second stage in saturation and avoiding nonlinearity. ADC performance is summarized in Table VII and Fig. 19.

B. Analog & Mixed-Signal Parts

To verify mixed-signal behavior, a scaled level 6 input at 6mV_{pp} was injected. As shown in Fig. 21(a), every 10 clock cycles, a switch is enabled to increase gain. If the maximum amplitude remains below the ADC's full range after 10 cycles, the gain increments again until the desired signal level is reached.

C. Digital

The Python simulation was used to produce realistic signals. At each step in the demodulation chain, it saved the inputs and outputs in .txt format, so at any moment it was possible to feed the Verilog model those signals. An API was developed to save the data into the correct binary format which Verilog recognizes. Then the comparison between output results from Verilog and the expected output values was made.

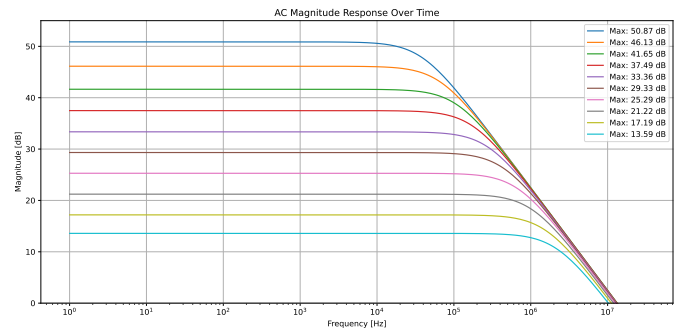


Fig. 18. Input vs. output VGA

Comparator	Ideal V_{tr} (mV)	Real V_{tr} (mV)	DNL	INL
1	75	114	N/A	N/A
2	150	154	-0.47	-0.47
3	225	233	0.05	-0.41
4	300	303	-0.07	-0.48
5	375	375	-0.04	-0.52
6	450	447	-0.04	-0.56
7	525	519	-0.04	-0.60
8	600	591	-0.04	-0.64
9	675	663	-0.04	-0.68
10	750	741	0.04	-0.64
11	825	819	0.04	-0.60
12	900	897	0.04	-0.56
13	975	969	-0.04	-0.60
14	1050	1047	0.04	-0.56
15	1125	1125	0.04	-0.52

TABLE VII
COMPARATOR TRIGGER VOLTAGES WITH CALCULATED DNL AND INL VALUES.

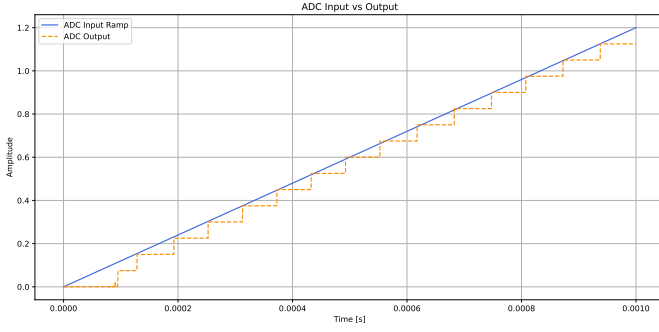


Fig. 19. Input (ramp) vs. output ADC

1) *IQ-Downmixer*: It was first tested separately block per block until combining them together and testing all at once. All the tests were conducted with the parameters listed in Table VIII.

The message "Hello world" was chosen as a sample message.

a) *Mixer*: from the Python script the sine, cosine and input signals were saved in .txt files and were fed to the Verilog code. After comparing, it was observed that they were similar.

b) *NCO*: To test the NCO, the .vcd files were used as a benchmark and a visual inspection could be conducted (as seen in the next paragraph).

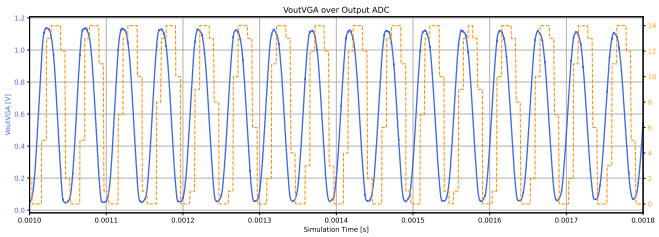


Fig. 20. Steady-state mixed-signal simulation, input: level 6, $6mV_{pp}$

Parameters	Value
OSR	16
$bit\ rate$	100
f_c	20 kHz
f_s	160 kHz

TABLE VIII
PARAMETERS FOR TESTING

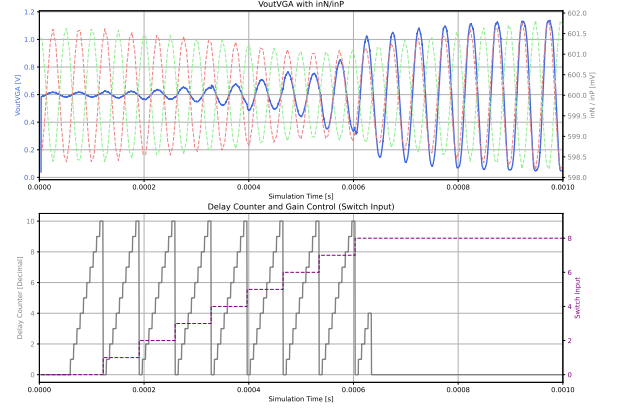


Fig. 21. Start of mixed-signal simulation, input: level 6, $6mV_{pp}$

c) *CIC filter*: With an ideal decimation ratio of 100, the CIC filter had promising results.

After successfully passing those small tests, a simulation between the mixer to the CIC filter was run. A lot of efforts were spent in debugging the CIC filter as it requires a good synchronization and each lines need to be monitored to avoid overflow error. A good structure and using generate was key to maintain a clean and readable code-base.

d) *Carrier Recovery*: The threshold values were manually adjusted, a cool-down of 15 clock cycles was the ideal trade-off to avoid changing the frequency too quickly and to let the result propagate through the receiving chain. Small adjustments were done to the NCO parameters resulting in a smooth centering of the signal around the 0 axis. No oscillation or issue even with '!' ('11111111' in varicode), which is the worst case scenario for the upper limit, was experienced. The same holds for the lower threshold worst case, ' ', which is '1', and ' ', which is '00100100100' in varicode.

Finally, noisy inputs were used to find the ideal lock time. The lock time is the amount of clock cycle where, if no changes to the frequency were made, it can be safely assumed that the algorithm converged. It was found that 2500 cycles was the right amount to lock the carrier search without being too sensitive to noise while ensuring that it remains between the threshold.

e) *Full IQ-Downmixer*: Finally, the last test was from the signal to the output of the CIC filter. The performance was efficient. However, at this time the carrier tracking couldn't be tested, as it requires the output of the next set of blocks.

The testing was done using an 8 bit NCO and a 4 bit NCO generator, but the difference in Mean Square Error (MSE) was quite small and the added overhead was acceptable.

	MSE Q	MSE R
NCO 4 bits	0.0308	0.0262
NCO 8 bits	0.0304	0.0278

TABLE IX
MSE COMPARISON

2) *GMSK-demodulation*: Like the IQ-downmixer, each sub-blocks were tested independently before running the full implementation.

a) *FIR filter*: First, the analysis of the FIR filter was conducted, as it could be inducing various errors. In section

III, the amount of taps is discussed, and it was decided to remove some below a certain threshold. This proved to be a good decision, as each tap is add extra overhead and removing some taps didn't really impact the model. To test this, the ideal Python model was used as an input, and the Register Transfer Level (RTL) output was compared to the ideal output:

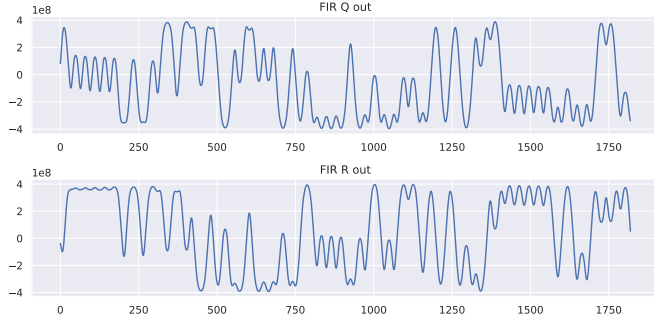


Fig. 22. Test of the FIR filter

b) *The Digital Part:* The same methodology as described was used for the two other sub-blocks. No anomaly or special behaviors were observed.

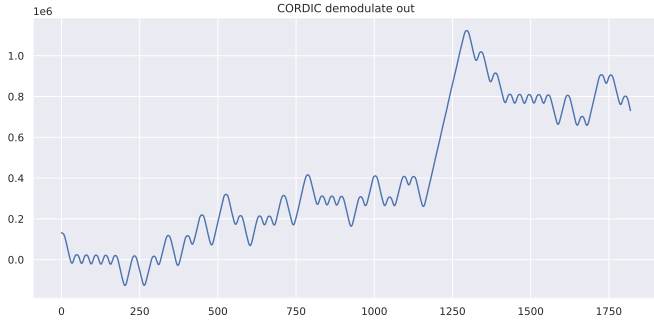


Fig. 23. Test of the other sub-blocks with ideal inputs

After testing and approving all individual components, it was time to launch the full test of the GMSK demodulation module. Compared to the ideal Python results, some ripples were observed at the peaks and valleys. This is most likely due to the *deriv* block and smaller bit precision. The derivation block will amplify this imprecision resulting in spikes.

However, this shouldn't impact the performance of the next block as those spikes stays in a restraint neighborhood.

3) *Clock recovery:* This block was somewhat easier to test, as it was not divided into multiple sub-blocks. First-order testing was done in the Verilog test-bench with an ideally generated demodulated signal sample. This was done to observe whether the block is functional in the way it needed to be.

The final testing was done in the following way, in the Python script: the demodulated signal was taken as input and the output of the clock recovery block was observed alongside the demodulated signal. The rising edges of the clock were supposed to be exactly at the peak values of the demodulated signal or in the close vicinity of them. This would mean the recovered clock triggers at appropriate points in time, which was naturally intended. In the end, the recovered clock was not as accurate as intended for higher noise environments - the example of the resulting characteristic may be seen on the figure 24.

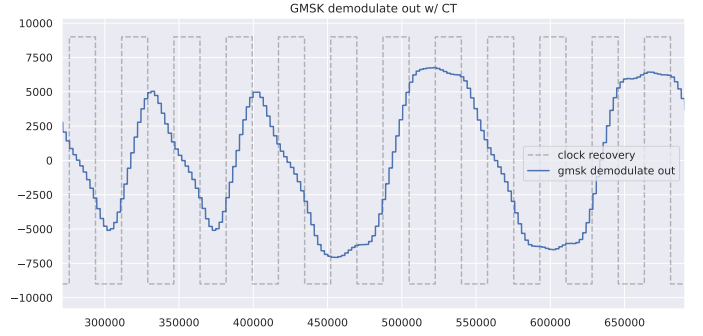


Fig. 24. Clock signal recovered from the GMSK-demodulated signal

4) *Carrier Recovery:* Using the script, the values were tweaked under various conditions until the results were acceptable. Ideally, those parameters should be dynamically calculated, but this approach requires extra hardware and logic, in other words it introduces additional complexity and power consumption.

5) *All together:* On top of the manual Python scripts, a tool was developed to simplify the digital tests. By simply running `python run.py`, it is possible to send a sample message. The user can send its own message, add noise, change the carrier frequency, use the proposed signals etc. More information can be found using `python run.py --help`. The digital part is working at level 6 with and without carrier tracking. It can also run at level 7, but with some difficulties - those results are in line with what was found in Fig. 2.

V. SYNTHESIS RESULTS

A. Digital

The last part is about preparing the design for tape-out and synthesizing. It is the moment of truth to know whether the design is actually realizable.

1) *Running:* Running the *genus* tool, it highlighted some design drawbacks. Typically, a lot of *packed arrays* were used [8], but the synthesis tool did not appreciate it very much. Hence, the instantiations had to be moved inside the reset sequence, and this was fairly synthesized.

The timing requirements were met:

$$f = 3.52 \text{ MHz} \quad t = 284.091 \text{ ns} \quad (9)$$

The clocks used all originated from the main and fastest one, as declared in `constraints.sdc`.

2) *Results:* This section introduces results from the synthesis - the categories which did not contribute to any metrics were removed.

Category	Leakage	Internal	Switching	Total [W]	Row%
register	$7.72e^{-6}$	$2.12e^{-5}$	$8.81e^{-7}$	$2.98e^{-5}$	55.74%
latch	$1.35e^{-8}$	$6.00e^{-8}$	$1.12e^{-8}$	$8.48e^{-8}$	0.16%
logic	$1.17e^{-5}$	$7.92e^{-6}$	$2.81e^{-6}$	$2.24e^{-5}$	41.94%
clock	0	0	$1.15e^{-6}$	$1.15e^{-6}$	2.15%
Subtotal	$1.94e^{-5}$	$2.92e^{-5}$	$4.85e^{-6}$	$5.35e^{-5}$	99.99
%	36.4	54.6	9.1	100	100

TABLE X
POWER REPORT FOR DIGITAL

With a staggering $53.5\mu W$, the digital part consumes 67 times less than expected.

The analog power consumption was higher than expected, the OTAs second stage needed more current to power the PGA

than originally assumed. The power consumption can be seen in Table XI.

Block	Power Consumption (μW)
PGA	181.32 μW
ADC	24.4 μW

TABLE XI
POWER CONSUMPTION PER ANALOG BLOCK

Instance	Module	Cell Count	Cell Area	Total Area
dgtop		13245	56021.56	56021.56
IQ_nco	NCO_WIDTH8	1771	4725.56	4725.56

TABLE XII
AREA REPORT FOR DIGITAL

As for the timing, the constraint is $284 \cdot 10^3$ ps, and the obtained critical path is of $4.449 \cdot 10^3$ ps so the requirements were met with a high margin.

3) *Discussion*: Significant leakage from logic gates, as detailed in Table X, suggests that implementing clock-gating or placing unused logic blocks in standby mode to reduce power consumption would be beneficial. Additionally, a high number of intermediate registers could be reduced to further obtain a lower leakage.

Another idea to reduce the maximum clock frequency and thus the energy, is to use a LUT-based NCO, which would allow operating at a lower clock frequency, thus decreasing overall power usage.

VI. RESULTS

Because the mixed-mode simulation is computationally intensive, where one second of transmission can take up to 16 hours, the test was performed on a reduced set of pwl input files. The analog-mixed signal chip was tested separately from the digital one to avoid any unforeseen crash or bug. Once the simulation completed successfully, the output from the ADC was exported and used as input to the digital section. This made it possible to track the carrier and decode a few characters reliably, up to level 6.

A. Noise performance



Fig. 25. Character error rate for various SNR

The performance of the system is a bit worse than expected. This result is partially due to the methodology used to find the error rate. If one character is appended, it will shift the rest of the string interpreting it as erroneous. So if the clock recovery ticks a bit faster and sends two spaces instead of one, it will degrade the rest of the decoding process. This can also be due to the limited precision of the FIR and NCO implementation. The best level reached was level 6.

B. Carrier Recovery

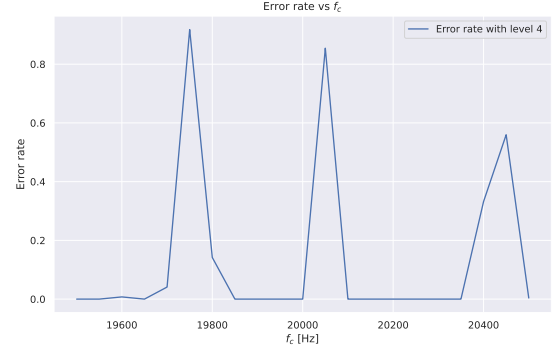


Fig. 26. Carrier recovery performance

The sudden spikes at some frequency are not due to issues with carrier recovery, but due to the clock recovery which does not always perform as expected (clocks at faulty moments).

If multiple tests were performed, this error will be more equally spread because this behavior is dynamic and depends on various factors, such as the noise, how fast the carrier recovery converges etc.

VII. CONCLUSION

a) *If I had one more month, I would change ?*: We wished we had more time to *iterate* over the design. The presented design met the specified requirements, but there wasn't much incentive to reach better performance.

In the analog part, the ADC topology would be changed from a Flash ADC to a SAR ADC. Flash ADCs are designed for high-speed applications, but the highest input frequency in this project is only $20.5 kHz$, making its speed unnecessary and its power consumption inefficient. This change would lower the power consumption and the area needed while maintaining performance.

For the mixed signal part, the number of registers could be optimized, as only the highest thermometer code bits matter in the control.

As for the digital part, a LUT-based NCO would have been a better choice and would have reduced the main clock frequency to $160 kHz$. Regarding the clock recovery block, it should have been tested to noisier environments sooner to ensure proper optimization on time and not to endanger the functionality of the overall system.

b) *If I would start over again I would ?*: Additional enhancements for the clock recovery would be taken into account earlier to derive a more proper implementation. For instance, the calculation of the timing error would be made more precise (it would take more samples into account and derive smoother error), additional filtering would be introduced etc., all to achieve more accurate timing and better performance for lower SNR.

REFERENCES

- [1] M. S. d. Deus, S. Y. C. Catunda, A. W. A. Soares, and D. R. Belfort, "A Review on Variable and Programmable Gain Amplifiers and Applications," *Journal of Integrated Circuits and Systems*, vol. 17, pp. 1–12, Apr. 2022.
- [2] S. Dutta, "Design of a Strong-Arm Dynamic-Latch based comparator with high speed, low power and low offset for SAR-ADC," Oct. 2022. arXiv:2209.07259 [physics].
- [3] "Matched filter," Feb. 2025. Page Version ID: 1275318975.
- [4] "Clock Recovery Primer," Jan. 2024.
- [5] J. F. Gusberti, "PLL-based Clock Recovery," Mar. 2022.

- [6] T. Guangyu, C. Wenbing, Z. Hancheng, C. Yiwen, T. Yang, and Y. Xuehui, "Research and Implementation of the Early-late Gate Bit Synchronization Algorithm in the Non-coherent Spread Spectrum System," in *2022 International Symposium on Networks, Computers and Communications (ISNCC)*, pp. 1–4, July 2022.
- [7] T. PD, "GENERAL MANUAL GMSK100 P&D," Mar. 2025.
- [8] Admin, "SystemVerilog Packed Arrays."
- [9] "Varicode," Dec. 2024. Page Version ID: 1261421642.